

竞赛实战技巧与调试对拍清单

适用范围：ICPC / CCPC / 蓝桥杯 / 团体程序设计天梯赛。默认主语言为 C++。

更新时间：2026-04-27。

这份文档补充“API 背诵”和“算法板子”之外的赛场工程技巧：怎么写模板、怎么处理 `long long`、怎么开调试、怎么对拍、怎么减少低级错误。目标是把代码写得更稳，而不是把模板写得更花。

三份文档的分工：

- `竞赛API背诵清单.md`：语言和库函数。
- `竞赛算法板子背诵清单.md`：算法模板和题型判断。
- 本文：写代码、调试、对拍、提交前检查和赛场协作。

1. 基础代码骨架

建议个人模板保持短小稳定：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

const int INF = 0x3f3f3f3f;
const ll LINF = (ll)4e18;
const int MOD = 1000000007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

原则：

- 模板越大，越容易在闭卷环境写错；把高频且稳定的东西放进去。
- 比赛现场不要临时换模板风格。
- 不要把所有算法代码默认塞进每题文件，容易误改、误用、拖慢阅读。
- 函数名和变量名可以短，但关键含义要稳定，例如 `n,m,g,dist,vis,fa,sz`。

2. 宏定义怎么用

宏适合缩短固定写法，不适合隐藏复杂逻辑。

推荐：

```
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

using ll = long long;
using vi = vector<int>;
using vll = vector<long long>;
```

可以用，但别滥用：

```
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
```

不推荐：

```
#define int long long
#define endl '\n'
#define max(a,b) ...
#define min(a,b) ...
```

原因：

- `#define int long long` 会影响 `main`、重载、第三方模板、数组内存、`bitset` 下标和某些库函数；大型代码里排错成本很高。
- `endl` 本来有刷新语义，强行宏替换会改变语义；平时直接写 `'\n'`。
- 自己宏掉 `min/max` 容易破坏 `std::min/std::max`，也可能重复求值。

更稳写法：

```
int n;
long long ans = 0;
vector<long long> a(n);
```

需要全局用 64 位时，优先显式使用 `ll`。只有非常短的思维题、确认没有库接口冲突时，才考虑 `#define int long long`，并且要写 `int32_t main()`。

3. long long 与溢出

3.1 什么时候必须用 long long

场景	建议
<code>n, m <= 2e5</code> , 边权/数值到 <code>1e9</code>	距离、答案、前缀和用 <code>long long</code> 。
两个 <code>int</code> 相乘可能超过 <code>2e9</code>	乘法前转成 <code>1LL * a * b</code> 。
组合数、方案数、取模中间值	取模前乘法用 <code>long long</code> 。
Dijkstra、Floyd、树上路径和	距离数组用 <code>long long</code> 。
面积、叉积、几何整数坐标	坐标到 <code>1e9</code> 时叉积可能到 <code>1e18</code> , 用 <code>long long</code> 或 <code>__int128</code> 。

常见错误:

```
int a = 1000000, b = 1000000;
long long x = a * b;          // 错, 乘法已经在 int 中溢出
long long y = 1LL * a * b;    // 对
```

左移也要注意:

```
long long x = 1LL << 40;      // 对
long long y = 1 << 40;        // 错, 1 是 int
```

3.2 INF 怎么设

```
const int INF = 0x3f3f3f3f;      // 约 1e9, 可 memset
const long long LINF = 4e18;     // long long 最短路常用
```

不要设成 `LLONG_MAX` 后直接加边权:

```
if (dist[u] != LINF && dist[v] > dist[u] + w) {
    dist[v] = dist[u] + w;
}
```

3.3 __int128

当 `long long` 乘法可能超过 `9e18`, 用 `__int128` 做中间计算。

```
using i128 = __int128_t;

bool le_mul(long long a, long long b, long long limit) {
    return (i128)a * b <= limit;
}

void print_i128(i128 x) {
    if (x < 0) {
        cout << '-';
        x = -x;
    }
    if (x >= 10) print_i128(x / 10);
    cout << char('0' + x % 10);
}
```

```
}
```

注意：__int128 是 GCC/Clang 扩展，不是标准 C++。ICPC/CCPC Linux 环境常可用；蓝桥杯和天梯赛要看编译器。

4. 取模技巧

```
long long norm(long long x, long long mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

long long add(long long a, long long b, long long mod) {
    return (a + b) % mod;
}

long long mul(long long a, long long b, long long mod) {
    return a * b % mod;
}
```

常见坑：

- C++ 负数 % 仍可能是负数。
- $a / b \% MOD$ 通常不等于 $a * inv(b) \% MOD$ ，只有模意义下且逆元存在时才能这么做。
- 费马逆元要求 MOD 是质数，且被除数与 MOD 互质。
- 多个模数不要混用全局 MOD ，函数最好传参。

5. 本地编译命令

Linux / WSL / MinGW 常用：

```
g++ -std=gnu++17 -O2 -pipe -Wall -Wextra -Wshadow -DLOCAL -o main main.cpp
```

更强的本地调试版：

```
g++ -std=gnu++17 -O0 -g -Wall -Wextra -Wshadow \
-DLOCAL -D_GLIBCXX_DEBUG \
-fsanitize=address,undefined \
-fno-omit-frame-pointer \
-o main main.cpp
```

比赛提交前再用接近评测的版本跑一次：

```
g++ -std=gnu++17 -O2 -pipe -Wall -Wextra -Wshadow -o main main.cpp
```

说明：

- `-Wall -Wextra -Wshadow` 能抓出不少变量遮蔽、未使用、可疑比较。
- `-D_GLIBCXX_DEBUG` 能检查部分 STL 误用，如越界迭代器、非法区间、错误比较器，但会显著变慢，也可能改变标准库容器 ABI；只适合本地。
- `-fsanitize=address,undefined` 可抓数组越界、use-after-free、未定义行为、部分整数溢出等；只适合本地。

- `-O0` 方便调试, `-O2` 更接近评测; 有些 UB 在不同优化级别表现不同, 所以两种都要跑。

Windows PowerShell 示例:

```
g++ -std=gnu++17 -O2 -Wall -Wextra -Wshadow -DLOCAL -o main.exe main.cpp
.\main.exe < input.txt
```

6. Debug 宏

调试输出走 `cerr`, 不要污染标准输出。

简单版:

```
#ifdef LOCAL
#define dbg(x) cerr << "[" << __LINE__ << "]" " << #x << " = " << (x) << '\n'
#else
#define dbg(x) ((void)0)
#endif
```

多变量版:

```
#ifdef LOCAL
void debug_out() { cerr << '\n'; }

template <class T, class... Args>
void debug_out(const T& x, const Args&... args) {
    cerr << x;
    if constexpr (sizeof...(args)) cerr << ", ";
    debug_out(args...);
}

#define debug(...) do { \
    cerr << "[" << __LINE__ << "]" " << #__VA_ARGS__ << " = "; \
    debug_out(__VA_ARGS__); \
} while (0)
#else
#define debug(...) ((void)0)
#endif
```

常用方式:

```
debug(n, m, ans);
dbg(dist[u]);
```

注意:

- 提交前确认本地没有定义 `LOCAL`。
- 不要依赖 `ONLINE_JUDGE` 一定存在; 不同 OJ 不一定定义。
- `cerr` 输出太多也会拖慢程序, TLE 时先关 debug。
- 调试容器时, 可以临时写 `print(vector)`, 不要把超长 debug 模板背成负担。

7. assert 与自检

```
#include <cassert>

assert(0 <= v && v < n);
assert((int)a.size() == n);
assert(gcd(x, mod) == 1);
```

适合检查：

- 下标范围。
- 图边端点是否合法。
- 二分判定函数是否单调。
- 流量、匹配、计数是否满足基本守恒。
- 构造题输出是否满足题意。

提交前可以保留轻量 `assert`，但如果评测环境可能触发或数据很大，建议关掉。关闭方式：

```
#define NDEBUG
#include <cassert>
```

更好的做法是把慢自检包在 `LOCAL` 中：

```
#ifdef LOCAL
void check_answer() {
    // O(n^2) validation
}
#endif
```

8. 对拍是什么

对拍就是让“待测解法”和“暴力/可信解法”在大量随机小数据上比较输出。

适用：

- 贪心想法不确定。
- 复杂数据结构容易写错。
- DP 转移边界多。
- 构造题需要验证输出合法。
- 图论题有很多特殊连通性情况。

不适用：

- 暴力解也很难写对。
- 输出不唯一且没有 checker。
- 随机生成器覆盖不到关键极端情况。

9. 三文件对拍

准备三个程序：

- `sol.cpp`：你的正式解法。
- `brute.cpp`：暴力但可信的解法。
- `gen.cpp`：随机数据生成器。

`gen.cpp` 示例：

```
#include <bits/stdc++.h>
```

```
using namespace std;

int main(int argc, char** argv) {
    int seed = argc > 1 ? atoi(argv[1]) : 1;
    mt19937 rng(seed);

    int n = uniform_int_distribution<int>(1, 8)(rng);
    cout << n << '\n';
    for (int i = 0; i < n; i++) {
        int x = uniform_int_distribution<int>(-10, 10)(rng);
        cout << x << " \n"[i + 1 == n];
    }
    return 0;
}
```

Bash 对拍脚本:

```
g++ -std=gnu++17 -O2 -o sol sol.cpp
g++ -std=gnu++17 -O2 -o brute brute.cpp
g++ -std=gnu++17 -O2 -o gen gen.cpp

for ((i=1; ; i++)); do
    ./gen $i > input.txt
    ./sol < input.txt > out1.txt
    ./brute < input.txt > out2.txt
    if ! diff -w out1.txt out2.txt; then
        echo "WA on seed $i"
        cat input.txt
        break
    fi
    echo "OK $i"
done
```

PowerShell 对拍脚本:

```
g++ -std=gnu++17 -O2 -o sol.exe sol.cpp
g++ -std=gnu++17 -O2 -o brute.exe brute.cpp
g++ -std=gnu++17 -O2 -o gen.exe gen.cpp

for ($i = 1; ; $i++) {
    .\gen.exe $i > input.txt
    .\sol.exe < input.txt > out1.txt
    .\brute.exe < input.txt > out2.txt
    cmd /c "fc /W out1.txt out2.txt > nul"
    if ($LASTEXITCODE -ne 0) {
        Write-Host "WA on seed $i"
        Get-Content input.txt
        break
    }
    Write-Host "OK $i"
}
```

如果输出包含浮点数, 不能直接 `diff`, 要写 checker。

10. 单文件对拍

把两个解法写进同一个文件，适合函数式题或小规模验证。

```
#include <bits/stdc++.h>
using namespace std;

int solve_fast(vector<int> a) {
    // TODO
    return 0;
}

int solve_slow(vector<int> a) {
    // TODO: brute force
    return 0;
}

int main() {
    mt19937 rng(1);
    for (int tc = 1; ; tc++) {
        int n = uniform_int_distribution<int>(1, 8)(rng);
        vector<int> a(n);
        for (int& x : a) x = uniform_int_distribution<int>(-5, 5)(rng);

        int x = solve_fast(a);
        int y = solve_slow(a);
        if (x != y) {
            cerr << "WA tc=" << tc << '\n';
            cerr << "n=" << n << '\n';
            for (int v : a) cerr << v << ' ';
            cerr << '\n';
            cerr << "fast=" << x << " slow=" << y << '\n';
            return 0;
        }
    }
}
```

优点是快，不需要脚本；缺点是不适合复杂输入输出和不同语言。

11. 随机数据生成技巧

只随机均匀数据不够，要混合生成：

数据类型	生成目标
最小数据	<code>n=1</code> 、空边、单点、无解。
最大附近小数据	对拍时让 <code>n</code> 接近暴力上限。
全相等	排序、去重、哈希、边界比较。
严格递增/递减	单调栈、双指针、LIS。
大量重复	map/set、压缩、计数。
负数和 0	前缀和、取模、除法、滑动窗口。
星形树/链形树	DFS 深度、树形 DP、LCA。
完全图/稀疏图	图复杂度和重边自环。
不连通图	最短路、MST、并查集。
极端权值	<code>INF</code> 、溢出、取模。

随机树生成：

```
int n = uniform_int_distribution<int>(1, 10)(rng);
cout << n << '\n';
for (int v = 2; v <= n; v++) {
    int p = uniform_int_distribution<int>(1, v - 1)(rng);
    cout << p << ' ' << v << '\n';
}
```

随机图生成：

```
int n = 6;
vector<pair<int,int>> edges;
for (int u = 1; u <= n; u++) {
    for (int v = u + 1; v <= n; v++) {
        if (uniform_int_distribution<int>(0, 1)(rng)) {
            edges.push_back({u, v});
        }
    }
}
cout << n << ' ' << edges.size() << '\n';
for (auto [u, v] : edges) cout << u << ' ' << v << '\n';
```

12. Checker 怎么写

输出唯一时，`diff -w` 通常够用。输出不唯一时，写 checker。

构造题 checker 示例：

```
bool check(const Input& in, const Output& out) {
    if ((int)out.p.size() != in.n) return false;
    vector<int> seen(in.n + 1);
    for (int x : out.p) {
        if (x < 1 || x > in.n || seen[x]) return false;
        seen[x] = 1;
    }
    return satisfies(in, out.p);
}
```

对拍时可比较：

- 两个输出的目标值是否相同。
- 输出方案是否都合法。
- 如果最优值唯一，比较最优值；方案不唯一则不要直接比字符串。

浮点 checker：

```
bool same(double a, double b) {
    return fabs(a - b) <= 1e-7 * max(1.0, max(fabs(a), fabs(b)));
}
```

13. 常见 WA 排查顺序

按这个顺序查，效率最高：

1. 重新读题，确认输入输出、下标、是否多组、是否有无解。
2. 手算样例，确认理解不是错的。
3. 检查边界：n=1、空集合、全相等、极值、不可达。
4. 检查类型：乘法前是否 1LL，距离是否 long long。
5. 检查初始化：多组数据是否清空，vis/dist/fa 是否重置。
6. 检查排序比较器：是否写了 <=，是否不满足严格弱序。
7. 检查容器删除：迭代器是否失效。
8. 检查二分：左右边界、更新方向、答案含义。
9. 检查图：有向/无向、重边、自环、编号从 0 还是 1。
10. 开 sanitizer 和 _GLIBCXX_DEBUG 跑小样例。
11. 写暴力和生成器对拍。

14. 常见 RE 排查顺序

1. 数组越界：数组大小是否包含 n+5，线段树是否 4*n。
2. 递归爆栈：链形树 DFS、深递归搜索。
3. 空容器取值：top/front/back/begin 前是否判空。
4. ms.erase(ms.find(x)) 前是否确认存在。
5. 除以 0 或模 0。
6. vector 下标是负数，尤其 i-1。
7. lower_bound 返回 end() 后是否解引用。
8. 函数漏 return。
9. 内存过大：二维数组、vector<vector<...>>、主席树节点数估错。

15. 常见 TLE 排查顺序

1. 复杂度是否估错，尤其多组测试总规模。

2. 是否在循环里复制大 `vector/string/map`。
3. DFS/BFS 是否重复入队或重复访问。
4. Dijkstra 是否忘记跳过过期状态。
5. `map/set` 是否可换成数组、离散化、`unordered_map`。
6. `endl` 是否频繁刷新。
7. debug 输出是否没关。
8. `vector` 是否反复扩容，可 `reserve`。
9. 递归记忆化是否真的命中。
10. 常数过大时考虑 `bitset`、前缀预处理、离线处理。

16. 二分和边界模板

求最小可行值：

```
long long l = 0, r = 1e18;
while (l < r) {
    long long mid = l + (r - l) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
```

求最大可行值：

```
long long l = 0, r = 1e18;
while (l < r) {
    long long mid = l + (r - l + 1) / 2;
    if (check(mid)) l = mid;
    else r = mid - 1;
}
```

浮点二分：

```
double l = 0, r = 1e9;
for (int it = 0; it < 100; it++) {
    double mid = (l + r) / 2;
    if (check(mid)) r = mid;
    else l = mid;
}
```

检查点：

- `check` 是否真的单调。
- 答案是否可能为负。
- `r` 是否一定可行，或 `l/r` 是否只是搜索范围。
- 输出 `l` 还是 `r`，整数二分结束后二者相等。

17. 比较器和排序

正确：

```
sort(a.begin(), a.end(), [](const Node& x, const Node& y) {
    if (x.l != y.l) return x.l < y.l;
    return x.r > y.r;
});
```

错误:

```
return x.l <= y.l; // 错, comp(x,x) 为 true
```

比较器必须满足严格弱序:

- `comp(x,x)` 必须是 `false`。
- 如果 `comp(a,b)` 为真, 则 `comp(b,a)` 必须为假。
- 不要在比较器里修改外部状态。
- 浮点排序不要用 `fabs(a-b)<eps` 乱判相等, 容易破坏传递性。

18. 迭代器失效

`vector` 中间插入/删除会让后面的迭代器失效:

```
for (auto it = v.begin(); it != v.end(); ) {
    if (*it < 0) it = v.erase(it);
    else ++it;
}
```

`set/map` 删除当前迭代器:

```
for (auto it = s.begin(); it != s.end(); ) {
    if (bad(*it)) it = s.erase(it);
    else ++it;
}
```

`priority_queue` 没有删除任意元素, 常用懒删除:

```
while (!pq.empty() && removed[pq.top().id]) pq.pop();
```

19. 多组数据清空

常见错误是上一组残留:

```
while (T--) {
    vector<vector<int>> g(n + 1);
    vector<int> vis(n + 1), deg(n + 1);
}
```

全局数组清空:

```
fill(head, head + n + 1, -1);
fill(dist, dist + n + 1, LINF);
for (int i = 1; i <= n; i++) g[i].clear();
```

时间戳技巧:

```
vector<int> tag(N);
int timer = 0;

timer++;
tag[x] = timer;
if (tag[y] == timer) {
    // visited in this round
}
```

适合多次清空大数组。

20. 输入输出技巧

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

注意：

- 关闭同步后不要混用 `cin/cout` 和 `scanf/printf`。
- 输出大量内容用 `'\n'`，不要用 `endl`。
- `getline` 前要吃掉上一行换行。
- 读到 EOF 用 `while (cin >> x)`。
- 交互题不要关闭必要刷新，输出后要 `cout << flush`。

格式控制：

```
cout << fixed << setprecision(10) << ans << '\n';
cout << setw(2) << setfill('0') << x << '\n';
```

21. 浮点技巧

```
const double EPS = 1e-9;

int sgn(double x) {
    if (fabs(x) < EPS) return 0;
    return x < 0 ? -1 : 1;
}
```

注意：

- 比较答案时看题目要求，通常输出 `1e-6` 或 `1e-9` 精度。
- 几何题的 `EPS` 不宜盲目固定，要结合坐标范围。
- `sqrt` 前如果可能出现 `-1e-12`，可以 `max(0.0, x)`。
- `long double` 可提高精度，但输出用 `cout` 更省心。
- 浮点数不要作为 `map/set` 的关键逻辑，除非离散化或转整数。

22. 位运算技巧

```
int lowbit(int x) { return x & -x; }

for (int s = mask; s; s = (s - 1) & mask) {
    // enumerate non-empty submasks
}

for (int s = mask; ; s = (s - 1) & mask) {
    // include zero
    if (s == 0) break;
}
```

常用内建函数：

```
__builtin_popcount(x);
__builtin_popcountll(x);
__builtin_ctz(x);      // x 不能为 0
__builtin_clz(x);      // x 不能为 0
```

注意：

- `1 << k` 中 `k >= 31` 会出问题，写 `1LL << k`。
- `__builtin_ctz(0)` 是未定义行为。
- 状压数组大小是 `1 << n`，当 `n >= 31` 不能用 `int`。

23. 哈希与随机化

字符串哈希：

```
using ull = unsigned long long;
const ull BASE = 1315423911ull;
```

`unordered_map` 防卡：

```
struct CustomHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
    }
};
```

注意：

- 哈希是概率正确，不是证明。
- 想降低碰撞概率，用双模或 `ull + mod` 双哈希。
- 随机化算法要固定可复现实验种子，调错时打印 `seed`。

24. 构造题自检

构造题不要只看样例，要写验证器：

```
bool valid(vector<int> p) {
    int n = p.size();
    vector<int> seen(n + 1);
    for (int x : p) {
        if (x < 1 || x > n || seen[x]) return false;
        seen[x] = 1;
    }
    return true;
}
```

常见不变量：

- 奇偶性。
- 总和是否固定。
- 度数和是否为 $2m$ 。
- 操作是否保持某个值不变。
- 图连通块数量变化。
- 排列每个数只能用一次。

构造失败时，优先找必要条件，再考虑是否充分。

25. 赛场 workflow

25.1 蓝桥杯 / 天梯赛

- 优先拿稳简单题，不要在第一时间追高难。
- 中文长题要标出输入范围、特殊输出、样例解释。
- 日期、模拟、字符串题先写小函数，避免主函数堆太长。
- 天梯赛有部分分思维，先拿可写的暴力/中档分，再优化。
- 个人赛没有队友复核，提交前自己读一遍输出格式。

25.2 ICPC / CCPC

- 三人一机，先按题目难度和实现风险分工。
- 复杂题先写思路和边界，别急着占机。
- 一人编码时，另一人准备样例、边界和对拍暴力。
- 模板题由最熟的人写，减少风格切换。
- 发生 WA 时，不要三个人同时盯同一行；一个人重读题，一个人造数据，一个人查代码。

26. 提交前 30 秒清单

- 文件名、语言、题号是否选对。
- 本地样例是否重新跑过。
- Debug 输出是否关闭。
- LOCAL、文件重定向是否移除。
- 多组数据是否清空。
- 数组大小是否够。
- long long 是否覆盖答案和中间乘法。
- 输出格式是否严格符合题意。
- 无解分支是否有输出。
- 构造题输出是否合法。

文件重定向建议这样写：

```

#ifdef LOCAL
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
#endif

```

不要在提交代码里留下无条件 `freopen`。

27. 个人模板建议

最小模板：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;

#define all(x) (x).begin(), (x).end()

#ifdef LOCAL
#define dbg(x) cerr << "[" << __LINE__ << "]" " << #x << " = " << (x) << '\n'
#else
#define dbg(x) ((void)0)
#endif

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}

```

可选增强：

- `print_i128`
- `CustomHash`
- `debug(...)`
- `chmin/chmax`
- 常用方向数组
- `ceil_div/floor_div`

`chmin/chmax`：

```

template <class T>
bool chmin(T& a, const T& b) {
    if (b < a) {
        a = b;
        return true;
    }
    return false;
}

template <class T>
bool chmax(T& a, const T& b) {
    if (a < b) {

```

```

    a = b;
    return true;
}
return false;
}

```

带负数也正确的整除：

```

long long floor_div(long long a, long long b) {
    assert(b != 0);
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return a / b;
    return - ((-a + b - 1) / b);
}

long long ceil_div(long long a, long long b) {
    assert(b != 0);
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return (a + b - 1) / b;
    return - ((-a) / b);
}

```

只处理非负数时可以用 $(a + b - 1) / b$ ；只要可能出现负数，就用上面的版本。

28. 内存和栈估算

常见大小：

类型	约占内存
<code>int</code>	4 B
<code>long long</code>	8 B
<code>double</code>	8 B
<code>pair<int,int></code>	8 B
<code>pair<long long,int></code>	通常 16 B，受对齐影响

估算方法：

```

int a[1000000]           约 4 MB
long long a[1000000]     约 8 MB
int seg[4 * 1000000]     约 16 MB
long long dp[2000][2000] 约 32 MB

```

注意：

- 大数组尽量开全局或 `static`，不要放在函数栈上。
- 链形树递归 DFS 可能爆栈；`n >= 2e5` 时要考虑迭代 DFS、BFS，或确认环境栈足够。
- `vector<vector<int>>` 有额外对象开销，超大图可改成扁平邻接表或链式前向星。
- 主席树、Trie、自动机要按“总节点数”估内存，不要只按 `n` 估。
- `bitset<N>` 是编译期大小；需要运行期大小时考虑 `vector<unsigned long long>` 或 `boost::dynamic_bitset`，但后者是否可用要看环境。

29. 赛规和外部工具纪律

训练时可以用搜索、题解、AI 工具做复盘，但正式比赛必须按当场规则执行。

常见要求：

- ICPC/CCPC：通常禁止电子资料、通讯工具和外部网络；是否允许纸质模板或 PDF TRD 看站规则。
- 蓝桥杯/天梯赛：通常按封闭个人赛准备，不要假设能访问互联网或本地资料。
- 校内选拔赛可能额外禁止访问非比赛网站、聊天工具和 AI/LLM 工具。

实战建议：

- 赛前确认允许使用的 IDE、编译器、语言标准、资料范围。
- 不把本地未授权模板、脚本、输入数据留在比赛机。
- 能打印纸质模板时，优先打印自己真正写过、测过、会改的版本。
- 不在比赛期间尝试规则边界；被判违规比 WA 严重得多。

30. 训练方式

每周至少做一次：

1. 找一道已 AC 的中等题，重新默写，不看旧代码。
2. 给这题写暴力和生成器，对拍 1000 组。
3. 用 sanitizer 跑一遍。
4. 把发现的坑写进个人模板注释。

每次赛前：

1. 用当前模板写一遍并查集、Dijkstra、线段树。
2. 用 PowerShell 或 Bash 跑一次对拍脚本。
3. 确认编译命令和 OJ 语言标准。
4. 清理模板里不熟悉、没验证过的代码。

31. 参考资料

- USACO Guide 调试清单： <https://usaco.guide/general/debugging-checklist>
- USACO Guide 基础调试与对拍： <https://usaco.guide/general/basic-debugging>
- GCC Warning Options： <https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html>
- GCC Instrumentation Options： <https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html>
- libstdc++ Debug Mode： https://gcc.gnu.org/onlinedocs/libstdc++/manual/debug_mode_using.html
- KACTL： <https://github.com/kth-competitive-programming/kactl>
- cppreference fundamental types： <https://cppreference.com/w/cpp/language/types>
- cppreference numeric_limits： https://www.cppreference.com/w/cpp/types/numeric_limits.html
- Codeforces 编译与调试实践： <https://mirror.codeforces.com/blog/entry/79024>
- Codeforces ONLINE_JUDGE 和常见 C++ 小技巧： <https://codeforces.com/blog/Redpo>